

Optimization Methods for Machine Learning and Engineering

Lecture 4 – Equality Constrained Optimization

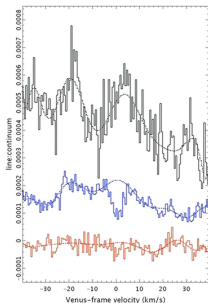
Julius Pfrommer



CC BY-SA 4.0

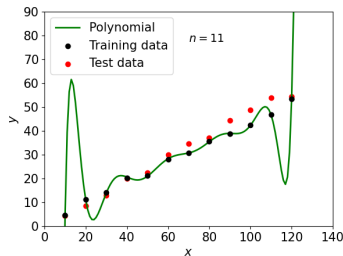
A Current Example for Overfitting

Life on Venus! But is it true?



PH3 spectra illustrating the **12th-order polynomial functions selected empirically** for fitting the spectral ripple seen with ALMA. [Greaves2020]

[Snellen2020] finds serious problems with the original analysis – that had been published in *Nature*!



Fitting a 12-th order polynomial to find small effects in comparatively noisy data is ... wild. Without taking precautions you are likely to be chasing phantoms.

Remember this quote from John von Neumann:
With four parameters I can fit an elephant, and with five I can make him wiggle his trunk. [Dyson2004]

Agenda

1. Some More Linear Algebra
2. Equality Constraints
3. Constraint Elimination
4. Lagrange Multipliers

Some More Linear Algebra

Solving Linear Equations

For $\mathbf{A} \in \mathbb{R}^{n \times m}$, $\mathbf{b} \in \mathbb{R}^n$ we look for $\mathbf{x} \in \mathbb{R}^m$ that solve the equation $\mathbf{Ax} = \mathbf{b}$

We can solve for $\mathbf{x}$ as $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ if $\mathbf{A}$ is invertible

The more robust way is to solve the optimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^m} \|\mathbf{Ax} - \mathbf{b}\| \quad (1)$$

Single solution Returns $\mathbf{x}$ with $\|\mathbf{Ax} - \mathbf{b}\| = 0$

Infinitely many solutions Most implementations select the solution with minimum norm $\|\mathbf{x}\|$

No solution Results in the solution minimizing the remaining error in the Euclidean norm.

Specialized solution techniques exist for different matrices $\mathbf{A}$ (sparse, upper/lower triangular, ...)

Example: We have two paint-colors: *Bordeaux* and *Indigo*. These contain different quantities of red and blue pigment.

Color	Bordeaux	Indigo
Blue Pigment	0.2	0.9
Red Pigment	0.8	0.1

How must we mix the paint to get *Violet* with a 50:50 pigment composition?

```
A = [0.2 0.9;
     [0.8 0.1]
b = [0.5; 0.5]
x = A \ b # Solves Equation (1)
# 2-element Array{Float64,1}:
#  0.5714285714285714
#  0.4285714285714286
```

Can we mix to get a paint with only blue pigment? Try out the computation! Is the solution realistic?

Determinants of a Matrix

Let $\mathbf{A}$ a square matrix from $\mathbb{R}^{n \times n}$. Its **determinant** $\det(\mathbf{A})$ is the **volume of the parallelogram** spanned by the column vectors $\mathbf{a}_i$.

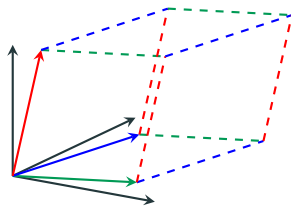
Rules for determinants:

- $\det(\mathbf{A}) \det(\mathbf{B}) = \det(\mathbf{AB})$
- $\det(\mathbf{A}) = \det(\mathbf{A}^\top)$
- $\det(\mathbf{A}^{-1}) = \det(\mathbf{A})^{-1}$

$\mathbf{A}$ **has an inverse** iff its **determinant is non-zero**.

The determinant is non-zero iff the column vectors $\mathbf{a}_i$ are **linearly independent**: There is no non-trivial way to linearly combine the $\mathbf{a}_i$ to form the null-vector (besides multiplying each vector with zero).

$$\{\mathbf{b} \in \mathbb{R}^n \mid \sum_{i=1}^n [b_i \mathbf{a}_i] = \mathbf{0}\} = \{\mathbf{0}\}$$



$$\mathbf{A} = \begin{pmatrix} | & | & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \mathbf{a}_3 \\ | & | & | \end{pmatrix}$$

using LinearAlgebra

```
A = [1 2;  
      3 4]
```

```
det(A) # -2.0
```

```
det(A*A) # 4.0
```

Definiteness of a Matrix

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ a symmetric matrix.

Positive Definiteness

$$\forall \mathbf{x} \in (\mathbb{R}^n \setminus \mathbf{0}), \mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$$

Positive Semi-Definiteness

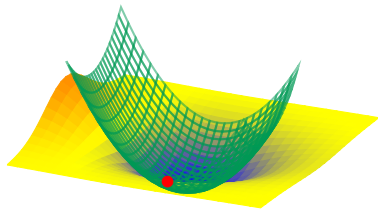
$$\forall \mathbf{x} \in (\mathbb{R}^n \setminus \mathbf{0}), \mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$$

We write $\mathbf{A} \succeq 0$ for a positive semi-definite $\mathbf{A}$.

If $\mathbf{A}$ is positive (semi-) definite, then $-\mathbf{A}$ is negative (semi-) definite.

The identity matrix $\mathbf{I}$ is positive definite.

Taylor Approximation with a Positive Definite Hessian



The Hessian is positive at the selected point $\mathbf{x}$.

But, as the depicted function is not everywhere convex, other points do not have a positive definite Hessian.

A **strictly convex twice differentiable** function has a **positive definite Hessian** matrix.

Basis, Rank and Nullspace

Suppose the column vectors $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^m$ are linearly independent. They are then a **basis** that spans a space of $n \leq m$ dimensions embedded in $\mathbb{R}^m$.

$$\{\mathbf{x} \in \mathbb{R}^m \mid \exists \mathbf{b} \in \mathbb{R}^n, \sum_{i=1}^n b_i \mathbf{a}_i = \mathbf{x}\}$$

For a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, there exists a basis of $k \leq \min(m, n)$ linearly independent column vectors that spans the space of all points reachable by $\mathbf{A}\mathbf{x}$ for $\mathbf{x} \in \mathbb{R}^n$.

The matrix is then said to have **rank** $k = \text{rank}(\mathbf{A})$.

There may be $\mathbf{y} \in \mathbb{R}^n$, $\mathbf{y} \neq \mathbf{0}$ with $\mathbf{A}\mathbf{y} = \mathbf{0}$. Linearly independent vectors with that property span the **nullspace**.

Rank-Nullity Theorem: The nullspace has dimension $n - k$.

```
using LinearAlgebra
```

```
A = [1 2 3 4;  
     5 6 7 8]
```

```
rank(A)
```

```
# 2
```

```
N = nullspace(A)
```

```
# 4x2 Array{Float64,2}:
```

```
# -0.400087 -0.374072
```

```
# 0.254633 0.796971
```

```
# 0.690996 -0.471724
```

```
# -0.545542 0.0488261
```

```
A * N[:,1]
```

```
# 2-element Array{Float64,1}:
```

```
# 0.0
```

```
# 0.0
```

Equality Constraints

Optimal Transport

The Kantorovich Problem [Kantorovich1942]

The red army general wants to move his troops from barracks to the frontline positions with minimal effort.

What is the most effective way of moving the soldiers so that all positions are filled?

The Hitchcock Problem [Hitchcock1941]

A farmers cooperative wants to deliver produce to different market locations. *What is the most effective way of moving the produce* so that the market demands are fulfilled?

Both are instances of the same *Optimal Transport Problem*. This class of problems has many practical applications.

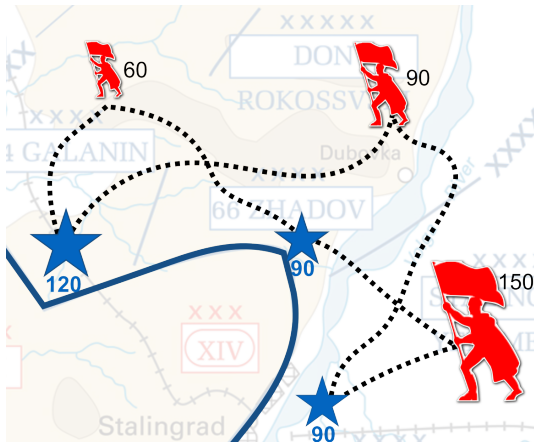


Image taken from Marco Cuturi

Optimal Transport 2

Initially, there are (60, 90, 150) thousand soldiers in the barracks. The fighting positions are laid out for (120, 90, 90) thousand soldiers.

The **number of soldiers moved** from barracks b_i to fighting positions p_j is expressed in the matrix $\mathbf{M}$.

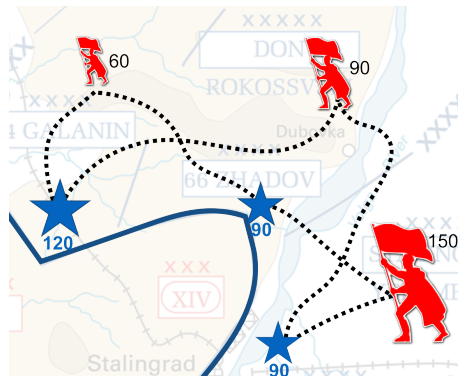
The **movement cost** between barracks and fighting positions depends on the distance between locations. The cost is expressed in the matrix $\mathbf{C}$.

$$\mathbf{M} = \begin{matrix} & \begin{matrix} p_1 & p_2 & p_3 \end{matrix} \\ \begin{matrix} b_1 \\ b_2 \\ b_3 \end{matrix} & \begin{pmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{pmatrix} \end{matrix}, \quad \mathbf{C} = \begin{matrix} & \begin{matrix} p_1 & p_2 & p_3 \end{matrix} \\ \begin{matrix} b_1 \\ b_2 \\ b_3 \end{matrix} & \begin{pmatrix} 1 & 3 & 5 \\ 3 & 1 & 7 \\ 5 & 3 & 1 \end{pmatrix} \end{matrix}$$

Set the 9 elements of the movement matrix at minimum cost so that all requirements are fulfilled.

Optimal Transport 3

$$\begin{aligned} \min_{\mathbf{M} \in \mathbb{R}^{3 \times 3}} \quad & \sum_{i,j \in \{1,2,3\}} m_{ij} c_{ij} \\ \text{subject to} \quad & \sum_{j=1}^3 m_{1,j} = 60 \quad \text{Soldiers moving out of } b_1 \\ & \sum_{j=1}^3 m_{2,j} = 90 \\ & \sum_{j=1}^3 m_{3,j} = 150 \\ & \sum_{i=1}^3 m_{i,1} = 120 \quad \text{Soldiers moving into } p_1 \\ & \sum_{i=1}^3 m_{i,2} = 90 \\ & \sum_{i=1}^3 m_{i,3} = 90 \\ & m_{ij} \geq 0, \quad i, j \in \{1,2,3\} \end{aligned}$$



This is a **Linear Program (LP) with equality constraints**.
Now we bring this optimization problem into **standard form**.

Optimal Transport 4

We write the **decision variables** in a **single column vector** $x = (m_{:,1}, m_{:,2}, m_{:,3})$.

Usually we want to **stack the column vectors of matrices**. (vec command in Julia)

Likewise, the **cost matrix as a stacked vector** is $\xi = (c_{:,1}, c_{:,2}, c_{:,3})$.

$$\begin{aligned} \min_{M \in \mathbb{R}^{3 \times 3}} \quad & \sum_{i,j \in \{1,2,3\}} m_{ij} c_{ij} \\ \text{subject to} \quad & \sum_{j=1}^3 m_{1,j} = 60 \\ & \sum_{j=1}^3 m_{2,j} = 90 \\ & \sum_{j=1}^3 m_{3,j} = 150 \\ & \sum_{i=1}^3 m_{i,1} = 120 \\ & \sum_{i=1}^3 m_{i,2} = 90 \\ & \sum_{i=1}^3 m_{i,3} = 90 \\ & m_{ij} \geq 0, \quad i, j \in \{1,2,3\} \end{aligned} \quad \Leftrightarrow \quad \begin{aligned} \min_{x \in \mathbb{R}^9} \quad & \xi^\top x \\ \text{subject to} \quad & \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix} x = \begin{pmatrix} 60 \\ 90 \\ 150 \\ 120 \\ 90 \\ 90 \end{pmatrix} \\ & x \geq 0 \end{aligned}$$

Equality Constraints must be Affine for Convexity!

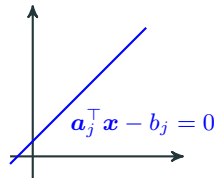
$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, l \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, m \end{aligned}$$

- Each **equality constraint must be affine** for the overall domain to be convex
- Individual affine equality constraints $h_j(\mathbf{x}) = 0$ can be written as $\mathbf{a}_j^\top \mathbf{x} - b_j = 0$
- The **combined equality constraints** are

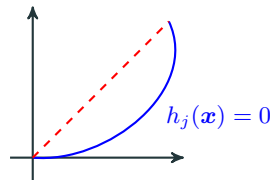
$$\boxed{\mathbf{A}\mathbf{x} = \mathbf{b}}$$

- The solutions are constrained to the **intersection of half-spaces** defined by $\mathbf{a}_j$, the rows of the matrix $\mathbf{A}$, and the scalars b_j

Affine (Convex) Equality Constraint



Non-Convex Equality Constraint



Constraint Elimination

Constraint Elimination

Suppose affine constraints in matrix form $\mathbf{A}x = b$ with $\mathbf{A} \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$

Let $k = \text{rank}(\mathbf{A}) < n$. Effectively there are **only** $n - k$ **degrees of freedom** for potential solutions

Can we **reduce the dimensionality** of the constrained optimization problem to $n - k$?

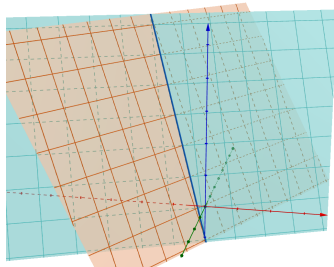
Let x_0 a feasible solution $\mathbf{A}x_0 = b$. There exists a matrix $\mathbf{F} \in \mathbb{R}^{n \times n-k}$ so that for all $z \in \mathbb{R}^{n-k}$

$$\mathbf{A}(\mathbf{F}z + x_0) = b \quad \text{and} \quad \{x \in \mathbb{R}^n : \mathbf{A}x = b\} = \{\mathbf{F}z + x_0 : z \in \mathbb{R}^{n-k}\}$$

Rewrite the optimization problem **in unconstrained form**: $z^* = \arg \min_{z \in \mathbb{R}^{n-k}} f(\mathbf{F}z + x_0)$

The solution to the original problem is reconstructed as $x^* = \mathbf{F}z^* + x_0$

Intersection of constraints $\Rightarrow$ 1D line



Constraint Elimination 2

For the matrix $\mathbf{F}$ we need the following property:

$$\begin{aligned}\mathbf{A}(\mathbf{F}\mathbf{z} + \mathbf{x}_0) &= \mathbf{b} \\ \mathbf{A}(\mathbf{F}\mathbf{z}) + \mathbf{A}\mathbf{x}_0 &= \mathbf{b} \\ \mathbf{A}(\mathbf{F}\mathbf{z}) &= \mathbf{0}\end{aligned}$$

So $\mathbf{F}$ has to *map any $\mathbf{z} \in \mathbb{R}^{n-k}$ into the nullspace* of $\mathbf{A}$.

Recall that the nullspace of $\mathbf{A}$ has $n - k$ linearly independent basis vectors.

$$\text{null}(\mathbf{A}) = \{\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_{n-k}\}$$

Construct the matrix $\mathbf{F}$:

$$\mathbf{F} = \begin{pmatrix} | & | & \cdots & | \\ \mathbf{d}_1 & \mathbf{d}_2 & & \mathbf{d}_{n-k} \\ | & | & & | \end{pmatrix}$$

The expression $\mathbf{F}\mathbf{z}$ returns a linear combination of the basis vectors of the nullspace of $\mathbf{A}$.

Solving the transport problem with constraint elimination

$$\begin{array}{ll}\text{minimize} & \boldsymbol{\xi}^\top \boldsymbol{x} \\ \text{subject to} & \underbrace{\begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}}_{\mathbf{A}} \boldsymbol{x} = \underbrace{\begin{pmatrix} 60 \\ 90 \\ 150 \\ 120 \\ 90 \\ 90 \end{pmatrix}}_{\mathbf{b}} \\ & \boldsymbol{x} \geq 0\end{array}$$

1. Find a **feasible solution for the equality constraint** with $\boldsymbol{x}_0 = \min_{\boldsymbol{y}} \|\mathbf{A}\boldsymbol{y} - \mathbf{b}\|_2$ and verify that indeed $\mathbf{A}\boldsymbol{x}_0 = \mathbf{b}$
2. Find a **basis for the null-space of $\mathbf{A}$** :

$$\text{null}(\mathbf{A}) = \{\boldsymbol{n}_1, \boldsymbol{n}_2, \dots\}$$

Recall: For the basis of the null-space the $\boldsymbol{n}_i$ are linearly independent and $\mathbf{A}\boldsymbol{n}_i = \mathbf{0}$

3. **Construct the matrix $\mathbf{F}$**
$$\mathbf{F} = \begin{pmatrix} | & | & \cdots \\ \boldsymbol{n}_1 & \boldsymbol{n}_2 & \\ | & | & \end{pmatrix}$$
4. The **unconstrained optimization problem** is

$$\begin{array}{ll}\min_{\boldsymbol{z} \in \mathbb{R}^{n-m}} & \boldsymbol{\xi}^\top (\mathbf{F}\boldsymbol{z} + \boldsymbol{x}_0) \\ \text{subject to} & \mathbf{F}\boldsymbol{z} + \boldsymbol{x}_0 \geq \mathbf{0}\end{array}$$

Solution in Julia

```
using LinearAlgebra
```

```
# Movement cost as a vector
```

```
xi = vec([1 3 5;  
          3 1 7;  
          5 3 1])
```

```
# Movement constraints
```

```
A = [1 0 0 1 0 0 1 0 0;  
      0 1 0 0 1 0 0 1 0;  
      0 0 1 0 0 1 0 0 1;  
      1 1 1 0 0 0 0 0 0;  
      0 0 0 1 1 1 0 0 0;  
      0 0 0 0 0 0 1 1 1]
```

```
b = [60, 90, 150, 120, 90, 90]
```

```
# 1. Initial admissible solution for  
# the equality constraint
```

```
x0 = A \ b
```

```
# 2. and 3. Construct F
```

```
F = nullspace(A)
```

```
# 9×4 Array{Float64,2}:
```

```
#  0.459815  -0.00663163  0.436229  0.206578  
#  0.186907  -0.0347273 -0.580929 -0.266131  
# -0.646722   0.0413589  0.144699  0.0595528  
# -0.261948  -0.562959 -0.242698  0.00176205  
# -0.0774331  0.392097  0.0935201  0.525321  
#  0.339381   0.170862  0.149177 -0.527084  
# -0.197867   0.56959  -0.193532 -0.20834  
# -0.109474  -0.35737  0.487409 -0.25919  
#  0.307341  -0.212221 -0.293877  0.467531
```

```
nx = size(F,1) # dimensions of the original problem
```

```
nz = size(F,2) # dimensions of the unconstrained problem
```

Solution in Julia

```
# Load the definitions for the
# Interior Point Method algorithm
include("optimization_library.jl")

# 4. Define the transformed problem
# with only inequality constraints
#
# 4.1 Objective function
obj(z) = xi' * (F * z .+ x0)
dobj(z) = (xi' * F)'
Hobj(z) = zeros(nz,nz)

# 4.2 Positivity inequality constraints
# x >= 0 for the original problem
gg = [z -> -(F[i,:]' * z + x0[i]) for i=1:nx]
dgg = [z -> -(F[i,:]) for i=1:nx]
Hgg = [z -> zeros(nz,nz) for i=1:nx]
```

```
# 5. Solve with the Interior Point Method.
# We are lucky. x0 > 0 is already admissible
# for the original problem. So we can start
# with zero as an interior point here.
z0 = zeros(nz)
(zstar, trace) = ipm(obj, dobj, Hobj, z0,
                    gg, dgg, Hgg)
xstar = F * zstar + x0

# Reshape and round for interpretation
round.(reshape(xstar, (3,3)))

# 3x3 Array{Float64,2}:
#  60.0  0.0  0.0
#  35.0 55.0  0.0
#  25.0 35.0 90.0

# Overall cost
xi' * xstar # => 540.0
```

Where to get help with these matrix calculus equations?

www.matrixcalculus.org



Matrix Calculus Documentation About

Matrix Calculus

MatrixCalculus provides matrix calculus for everyone. It is an online tool that computes vector and matrix derivatives (matrix calculus).

derivative of

$\mathbf{x}^T \mathbf{A} \mathbf{x}$

w.r.t.

$\mathbf{x}$

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^T \cdot \mathbf{A} \cdot \mathbf{x}) = \mathbf{A} \cdot \mathbf{x} + \mathbf{A}^T \cdot \mathbf{x}$$

where

$\mathbf{A}$ is a

matrix

$\mathbf{x}$ is a

vector

Export functions as

Python

Latex

Common subexpressions

ON



The Matrix Cookbook

2.4.4 Gradient and Hessian

Using the above we have for the gradient and the Hessian

$$\mathbf{f} = \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x} \quad (96)$$

$$\nabla_{\mathbf{x}} \mathbf{f} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^T) \mathbf{x} + \mathbf{b} \quad (97)$$

$$\frac{\partial^2 \mathbf{f}}{\partial \mathbf{x} \partial \mathbf{x}^T} = \mathbf{A} + \mathbf{A}^T \quad (98)$$

Contains many identities for “copy/paste”. But a general understanding is required.

A lot of the material is much more advanced than this course.

The Matrix Cookbook can be found on the shelf of many grad students in CS.

Lagrange Multipliers

Equality Constraints and Lagrange Multipliers

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^2} \quad & f(\mathbf{x}) = x_1^2 + 3x_2^2 \\ \text{subject to} \quad & x_1 + 2x_2 - 1 = 0 \end{aligned}$$

The **Lagrangian** $\mathcal{L}$ of the optimization problem is

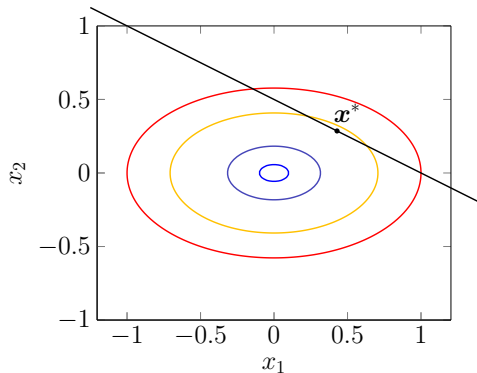
$$\mathcal{L}(\mathbf{x}, \lambda) = x_1^2 + 3x_2^2 + \lambda(x_1 + 2x_2 - 1)$$

Unconstrained: Optimum at $\nabla f = \mathbf{0}$

Constrained: Optimum at $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{0}$ and $\nabla_{\lambda} \mathcal{L} = 0$

Analytic Solution

$$\begin{aligned} \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda) &= \begin{pmatrix} 2x_1 + \lambda \\ 6x_2 + 2\lambda \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} & \Rightarrow & 2x_1 = 3x_2 = -\lambda \\ \nabla_{\lambda} \mathcal{L}(\mathbf{x}, \lambda) &= x_1 + 2x_2 - 1 = 0 & \Rightarrow & x_1 = \frac{3}{7}, x_2 = \frac{2}{7}, \lambda = -\frac{6}{7} \end{aligned}$$



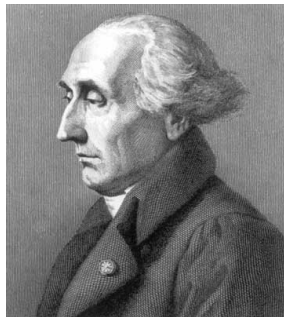
The Origin of Lagrange Multipliers [Lagrange1788]

The value of [Lagrange's book "Mécanique Analytique"] consists in the exposition of a general method by which every mechanical question may be stated in a single algebraic equation. The entire history of any mechanical system, as for example, the solar system, may thus be condensed into a single sentence.

Robert S. Woodward [Woodward1895]

Lagrange stated his principle in the more general setting of **variational analysis** where the optimization results are themselves functions. We will see a glimpse of this in Lecture 7 on **vector spaces**.

Lagrange multipliers were invented ~150 years before the introduction of digital computers. Many fields (e.g. physics) have relied on **analytic solutions** before numerical methods could be practically applied on high-dimensional problems.



Joseph-Louis Lagrange (1736-1813)
(Giuseppe Lodovico Lagrangia)

Lagrange Multipliers for Multiple Equality Constraints

Suppose the problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x})$$

subject to $\mathbf{Ax} = \mathbf{b}$

- With m equality constraints, so $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$
- The Lagrangian is $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top (\mathbf{Ax} - \mathbf{b})$ with an ***m -dimensional Lagrange multiplier $\boldsymbol{\lambda}$***
- The ***gradient and Hessian of the Lagrangian*** are

$$\nabla \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \begin{pmatrix} \nabla f(\mathbf{x}) + \mathbf{A}^\top \boldsymbol{\lambda} \\ \mathbf{Ax} - \mathbf{b} \end{pmatrix}, \quad \nabla^2 \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \begin{pmatrix} \nabla^2 f(\mathbf{x}) & \mathbf{A}^\top \\ \mathbf{A} & \mathbf{0} \end{pmatrix}$$

The constrained problem is ***transformed into an $(n + m)$ -dimensional unconstrained problem.***

But $\mathcal{L}$ is no longer convex! The solution $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ with $\nabla \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \mathbf{0}$ is the only ***saddle-point.***

The ***Newton-Method still applies*** to find stationary points. But since we search for a saddle-point (instead of minimizing $\mathcal{L}$), unmodified Gradient Descent and Line-Search can no longer be used.

Solving the Lagrangian Numerically

```
# Load the definitions for the
# Newton Method algorithm
include("optimization_library.jl")

# Objective Function, Gradient and Hessian
f(x) = x[1]^2 + 3*x[2]^2
df(x) = [2*x[1];
         6*x[2]]
Hf(x) = [2 0;
        0 6]

# Equality Constraints  $A*x = b$ 
A = [1 2]
b = [1]

# Dimensionality of x and lambda
nx = size(A,2)
nlambda = size(A,1)
```

```
# Define the Lagrangian and its derivatives
L(z) = begin
    x = z[1:nx]; lambda = z[nx+1:nx+nlambda]
    return f(x) + lambda' * (A*x-b)
end

dL(z) = begin
    x = z[1:nx]; lambda = z[nx+1:nx+nlambda]
    return vcat(df(x) + A' * lambda, A*x-b)
end

HL(z) = begin
    x = z[1:nx]; lambda = z[nx+1:nx+nlambda]
    return vcat(hcat(Hf(x), A'), hcat(A, zeros(nlambda,nlambda)))
end

# Solve with Newton's Method (w/o line-search!)
z0 = zeros(nx+nlambda)
zstar,trace = newton_descent(L, dL, HL, z0, ls=false)
# 3-element Array{Float64,1}:
#  0.4285714285714286  => x_1
#  0.28571428571428575  => x_2
# -0.8571428571428572  => lambda
```

Summary of what you learned today

- Some more Linear Algebra: **Rank** and **Nullspace**
- **Equality constraints** and the **Optimal Transport Problem**
- **Solving problems with equality constraints** by
 - **Reduction of the problem dimensionality** (Constraint Elimination)
 - **Increasing the problem dimensionality** (Lagrange Multipliers)

Next lecture: How to **combine Lagrange Multipliers with the Interior Point Method** in a way that scales to interesting problems

Also: Many application examples

- Mechanical Design
- Optimal Control
- Optimal Stock Investments

- [Dyson2004] Freeman Dyson. “A meeting with Enrico Fermi”. In: *Nature* 427 (2004), pp. 297–297.
- [Greaves2020] Jane S Greaves et al. “Phosphine gas in the cloud decks of Venus”. In: *Nature Astronomy* (2020), pp. 1–10.
- [Hitchcock1941] Frank L Hitchcock. “The distribution of a product from several sources to numerous localities”. In: *Journal of mathematics and physics* 20.1-4 (1941), pp. 224–230.
- [Kantorovich1942] Leonid Vitalievich Kantorovich. “On the translocation of masses”. In: *Dokl. Akad. Nauk. USSR (NS)*. Vol. 37. 1942, pp. 199–201.
- [Lagrange1788] Joseph-Louis Lagrange. *Mécanique Analytique*. 1788.
- [Snellen2020] IAG Snellen et al. “Re-analysis of the 267-GHz ALMA observations of Venus: No statistically significant detection of phosphine”. In: *arXiv preprint arXiv:2010.09761* (2020).
- [Woodward1895] Robert Simpson Woodward. “An Historical Survey of the Science of Mechanics”. In: *Science* 1.6 (1895), pp. 141–157.